

Trabajo Práctico Integrador: Compresor de imágenes RLE

Programación II

Ingeniería en Computación (UNRN)

Enrique Nicanor Mariotti (mariottien@gmail.com)

Junio 2026

Resumen

El propósito de este trabajo es desarrollar un programa que pueda comprimir y descomprimir imágenes. El mismo recibe un archivo con una imagen en un formato estándar (*.bmp*), y genera un nuevo archivo de tamaño menor o igual al original. A su vez, si el programa recibe un archivo con una imagen en el formato propio (*.prle*), genera un archivo en el formato estándar original (*.bmp*), sin presentar pérdidas de información. La solución plateada se enfoca en el caso de las imágenes que tienen fondo plano o transparente (es decir, con muchos píxeles que tienen el mismo valor). Los requerimientos para la implementación de la misma fueron que el algoritmo pueda manejar imágenes grandes de hasta 2048×2048 y que la ejecución este en el orden de los segundos.

1. Introducción

Se describe aquí el diseño e implementación de un compresor de imágenes color *.bmp* de 24 bits basado en *Run-Length Encoding* (RLE) planar [Tamayo, 2005], con ejecución concurrente simple.

El RLE es una técnica de compresión sin pérdida que reemplaza secuencias de valores repetidos (*runs*) por un par (cantidad, valor), reduciendo significativamente el tamaño de datos con zonas uniformes. Su eficiencia depende de la frecuencia y longitud de esas secuencias: imágenes con fondos planos o áreas de color sólido obtienen ratios muy elevados, mientras que imágenes con ruido aleatorio pueden experimentar expansión.

La variante implementada combina dos decisiones de diseño principales. En primer lugar, la separación planar: en lugar de operar sobre los píxeles intercalados en formato BGR, como los almacena el *.bmp*, la imagen se descompone en tres planos independientes (R, G, B) antes de comprimir.

Esto maximiza la longitud de los runs en zonas de color uniforme, ya que los *runs* se pueden presentar de manera independiente en cada canal, y son mas probables que los *runs* de los 3 canales simultáneamente.

En segundo lugar, la concurrencia: al no existir dependencias entre los tres canales, cada uno es comprimido en un hilo independiente, reduciendo el tiempo

de procesamiento en arquitecturas que lo soporten.

El formato de salida recibe la extensión *.prle* (*Planar RLE*) y es específico de este proyecto.

2. Formato

2.1. Header

El archivo comienza con un header de 49 bytes en orden little-endian:

Offset	Tamaño	Tipo	Campo
0	4	uint32	Identificador (0xCAFECAFE)
4	1	uint8	Versión (0x01)
5	4	uint32	Ancho de la imagen en píxeles
9	4	uint32	Alto de la imagen en píxeles
13	8	uint64	Offset del canal R dentro del archivo
21	4	uint32	Tamaño comprimido del canal R en bytes
25	8	uint64	Offset del canal G dentro del archivo
33	4	uint32	Tamaño comprimido del canal G en bytes
37	8	uint64	Offset del canal B dentro del archivo
45	4	uint32	Tamaño comprimido del canal B en bytes

El identificador permite detectar archivos inválidos antes de intentar la decodificación. Los offsets permiten acceder a cada canal de forma independiente en tiempo $\mathcal{O}(1)$ sin recorrer el archivo completo. Los tamaños exactos de cada canal eliminan la necesidad de marcadores de fin de stream de datos.

A continuación del header se almacenan los datos comprimidos de los tres canales en forma contigua:

$$\underbrace{\text{Header}}_{49 \text{ B}} \mid \underbrace{\text{Datos R}}_{\text{size}_r \text{ B}} \mid \underbrace{\text{Datos G}}_{\text{size}_g \text{ B}} \mid \underbrace{\text{Datos B}}_{\text{size}_b \text{ B}}$$

Aquí, los canales se tratan como un vector unidimensional de datos de dimensión $\text{width} \times \text{height}$, donde los píxeles se leen desde la esquina superior izquierda de la imagen, de izquierda a derecha, y de arriba hacia abajo.

Notar que el orden de los canales es el inverso al usado en el formato *.bmp* [Liesch, 2003].

2.2. Tokens

Cada canal comprimido es un stream de tokens [James D. Murray, 2006]. El tipo de cada token queda determinado por los dos bits más significativos del byte de control (byte menos significativo):

Bits 7..6	Tipo	Formato	Rango de count
00	Run corto	[00 ccccc] [value]	$1 \leq n \leq 63$
01	Run largo	[01 CCCCC] [ccccccc] [value]	$64 \leq n \leq 16447$
10	Literal corto	[10 ccccc] [$v_0 \dots v_N$]	$1 \leq n \leq 63$
11	Literal largo	[11 CCCCC] [ccccccc] [$v_0 \dots v_N$]	$64 \leq n \leq 16447$

Los tokens cortos usan los 6 bits restantes del byte de control directamente como count. Los tokens largos extienden el count con un byte adicional, formando un campo de 14 bits con un desplazamiento (*offset*) de +64:

$$\text{count} = (\text{CCCCC} \ll 8 \mid \text{ccccccc}) + 64$$

Este desplazamiento permite que el rango largo comience exactamente donde termina el corto, sin solapamiento ni desperdicio de códigos.

La existencia de dos tipos de elementos **run** y **literal** permite no penalizar áreas donde existe mucha variación con run de largo 1, lo que añade información irrelevante a la codificación. Por otro lado, existencia de elementos cortos y largos ahorra 1 byte en la codificación de los tokens, dependiendo del largo de los mismos.

2.3. Interpretación

Un token del tipo **run** indica que el valor **value** se repite exactamente **count** veces en el stream de salida. El decoder lo expande escribiendo **count** copias del byte.

Un token de **literal** indica que los **count** bytes que siguen al encabezado son valores distintos que se copian directamente al stream de salida sin modificación.

2.4. Restricciones

- **count** = 0 no esta incluido en la codificación, ya que los run largos presentan un *offset* de +64 píxeles. En si mismo un **count** = 0 no tiene significado definido.
- Runs mayores a 16447 píxeles se codifican como múltiples tokens de run consecutivos con el mismo valor.
- El tamaño exacto de cada canal descomprimido se conoce de antemano a partir de **width** × **height**, por lo que no se necesita ningún token de fin de canal.

3. Algoritmo de codificación

La función `compress_channel()` recibe el plano de un canal como un arreglo lineal de bytes (un byte por píxel) y produce el stream comprimido correspondiente.

3.1. Estructuras auxiliares

- **out**: Buffer de salida donde se escriben los tokens generados.
- **lit_buf**: Buffer acumulador de píxeles literales. Se vuelca a **out** como un token literal al detectar un run o al final del stream.
- **flush_literal()**: Función auxiliar que emite el contenido de **lit_buf** como un token literal y lo vacía. No realiza ninguna acción si **lit_buf** está vacío.

3.2. Criterio de codificación

Un run de longitud n es *rentable* si el ahorro en bytes respecto a emitirlo como literal es positivo.

Son particularmente de interés, las secuencias de datos de menor longitud, ya que en las mismas es donde se presentan los casos menos rentables.

Con el formato definido:

- Run corto: 2 bytes fijos independientemente de n (para $1 \leq n \leq 63$).
- Literal corto de n bytes: $1 + n$ bytes (para $1 \leq n \leq 63$)

3.2.1. Tratamiento del run de longitud 1

En cualquier caso, codificar 1 píxel produce la misma longitud ya sea si se codifica como run o literal. En esta implementación elegimos hacerlo como literal, de manera tal de que los literales pueden concatenarse si fuera el caso.

3.2.2. Tratamiento del run de longitud 2

Un run de 2 píxeles ocupa 2 bytes como run, pero 3 bytes como literal, por lo que a primera vista siempre sería ideal codificarlo como run.

Sin embargo, si existe un bloque de literales abierto en **lit_buf**, cerrarlo para emitir el run introduce un token de control adicional. En ese caso es preferible absorber los 2 píxeles en el literal abierto:

$$\underbrace{\text{literal}(N) + \text{run}(2) + \text{literal}(M)}_{(1+N)+2+(1+M) \text{ bytes}} > \underbrace{\text{literal}(N+2+M)}_{1+(N+2)+M \text{ bytes}}$$

Si en cambio no hay literal abierto, el run de 2 se emite directamente como run (2 bytes versus los 3 bytes que costaría un nuevo literal).

3.2.3. Tratamiento del run de longitud 3

En cualquier caso, codificar 3 píxeles o mas produce menor longitud si se tratan como run que como literales.

Algorithm 1 COMPRESS_CHANNEL(in, L)

```
1:  $out \leftarrow$  buffer vacío
2:  $lit\_buf \leftarrow$  buffer vacío
3: function FLUSH_LITERAL
4:   if  $lit\_buf$  no está vacío then
5:     EMIT_LITERAL( $out, lit\_buf$ )
6:     vaciar  $lit\_buf$ 
7:   end if
8: end function
9:  $i \leftarrow 0$ 
10: while  $i < L$  do ▷ longitud del canal
11:   ▷ Medir el run que comienza en la posición  $i$ 
12:    $v \leftarrow in[i]$ 
13:    $j \leftarrow i + 1$ 
14:   while  $j < L$  y  $in[j] = v$  do
15:      $j \leftarrow j + 1$ 
16:   end while
17:    $n \leftarrow j - i$  ▷ longitud del run
18:   if  $n \geq 3$  then ▷ run  $n \geq 3$ 
19:     FLUSH_LITERAL()
20:     EMIT_RUN( $out, n, v$ )
21:   else if  $n = 2$  then ▷ run de  $n = 2$ 
22:     if  $lit\_buf$  está vacío then
23:       EMIT_RUN( $out, 2, v$ )
24:     else
25:       ▷ Absorber en literal abierto evita un token extra
26:       ▷ Tener cuidado de no desbordar  $lit\_buf$ 
27:       agregar  $v$  a  $lit\_buf$ 
28:       if  $|lit\_buf| = \text{LITERAL\_LONG\_MAX}$  then
29:         FLUSH_LITERAL
30:       end if
31:       agregar  $v$  a  $lit\_buf$ 
32:       if  $|lit\_buf| = \text{LITERAL\_LONG\_MAX}$  then
33:         FLUSH_LITERAL
34:       end if
35:     end if
36:   else ▷ literal de  $n = 1$ 
37:     agregar  $v$  a  $lit\_buf$ 
38:     if  $|lit\_buf| = \text{LITERAL\_LONG\_MAX}$  then
39:       FLUSH_LITERAL
40:     end if
41:   end if
42:    $i \leftarrow j$ 
43: end while
44: FLUSH_LITERAL ▷ agregar literales pendientes al final
45: return  $out$ 
```

3.3. Consideraciones

- La función `lit_buf` nunca supera `LITERAL_LONG_MAX` bytes entre dos instrucciones consecutivas, el flush se realiza inmediatamente después de cada `push` que alcanza el límite.
- Considerando el offset de +64, `count = 0` nunca es emitido, el run mínimo es 2 y el literal mínimo es 1, ambos detectados por las condiciones de guarda en `emit_run()` y `emit_literal()`.
- Todo byte del stream de entrada aparece exactamente una vez en el stream de salida, ya sea como parte de un run o de un literal.

4. Análisis de casos representativos

La tabla siguiente resume el costo en bytes de salida para distintos patrones de entrada para ilustrar diversos escenarios que pueden encontrarse:

Secuencia	Bytes de entrada	Bytes de salida	Ratio
A A A A (run ≥ 3)	4	2	2
A B C D (literales)	4	5	0,8
A A B B (run = 2)	4	4	1
A B B C (run = 2, entre literales)	4	5	0,8
A A B C C (literal, entre run = 2)	5	6	0,83

Nótese que cuando hay un literal abierto al encontrar un run de 2, en ese contexto es absorbido, y de esa manera existe un ahorro de 1 byte.

Mejor caso. Canal completamente uniforme (fondo plano de `width` $\times$ `height` píxeles con el mismo valor). El stream entero se codifica como una sucesión de runs largos de 16447 píxeles. Para una imagen de 2048×2048 :

$$\left\lceil \frac{4\,194\,304}{16\,447} \right\rceil = 256 \text{ tokens} \times 3 \text{ bytes} = 768 \text{ bytes} \Rightarrow \text{ratio} \approx 5461:1$$

Esta compresión solo aplica a un solo canal plano, sin considerar tampoco el overhead de cada formato.

Peor caso. Canal con cadenas literales largas, ya sea porque no hay píxeles iguales consecutivos, o porque hay runs de largo 2 absorbidos por las cadenas de literales. Para una imagen de 2048×2048 :

$$\left\lceil \frac{4\,194\,304}{16\,447} \right\rceil = 256 \text{ tokens} \times (16\,447 + 2) \text{ bytes} = 4\,210\,944 \text{ bytes} \Rightarrow \text{ratio} \approx 0,99:1$$

5. Resultados

La Fig.1 presenta algunos casos de pruebas en imágenes de 2048×2048 . Los resultados del algoritmo, considerando todos los canales y los respectivos headers del formato se resumen a continuación:

Test	Bytes entrada	Bytes salida	Ratio
Test a	12 582 966	2 353	5347
Test b	12 582 966	2 353	5347
Test c	12 582 966	12 584 490	0,99
Test d	12 582 966	125 124	100
Test e	12 582 966	1 050 161	12
Test f	12 582 966	2 353	5347
Test g	12 582 966	12 584 497	0,99
Test h	12 582 966	12 584 497	0,99
Test i	12 582 966	5 033 221	2,5

6. Conclusiones

El compresor desarrollado demuestra que puede alcanzar ratios de compresión elevados en el dominio para el que fue concebido; imágenes de gran tamaño con extensas zonas de color uniforme. La separación planar es una decisión de gran impacto sobre el ratio final, ya que transforma el problema de comprimir tripletes, donde la probabilidad de repetición es baja, en el de comprimir tres streams independientes de un solo canal, donde la probabilidad de repetición es mayor. En cuanto al rendimiento, el modelo de tres hilos independientes sobre los canales R, G y B es naturalmente apto para la tarea ya que hay ausencia de dependencias entre canales.

El formato de token de cuatro tipos, con count variable de uno o dos bytes, resuelve el principal problema de la expansión en zonas de alta variabilidad. Los tokens literales permiten transportar secuencias de píxeles distintos con un overhead de uno o dos bytes de control por bloque, manteniendo la expansión máxima muy por debajo del doble del tamaño original incluso en el peor caso teórico. Los resultados sobre imágenes generadas, corroboran las estimaciones teóricas. La reserva del valor `count = 0` en los formatos cortos, preserva la posibilidad de extender el formato sin romper la compatibilidad con streams existentes.

Como trabajo futuro, la incorporación de partición por bloques permitiría aumentar el grado de paralelismo más allá de los tres canales, mejorar la localidad de caché en imágenes muy grandes y habilitar la descompresión por región. Otros tipos de compresión se pueden realizar posteriormente a la aplicación de este algoritmo, para obtener tamaños aun menores. Dotar al formato de una comprobación de tipo CRC permitiría incorporar la comprobación de integridad de datos del archivo comprimido en tiempo de ejecución, con un costo extra muy bajo. Adicionalmente, la elección dinámica del recorrido optimo para maximizar la compresión podría mejorar significativamente los resultados de la Sección 5.

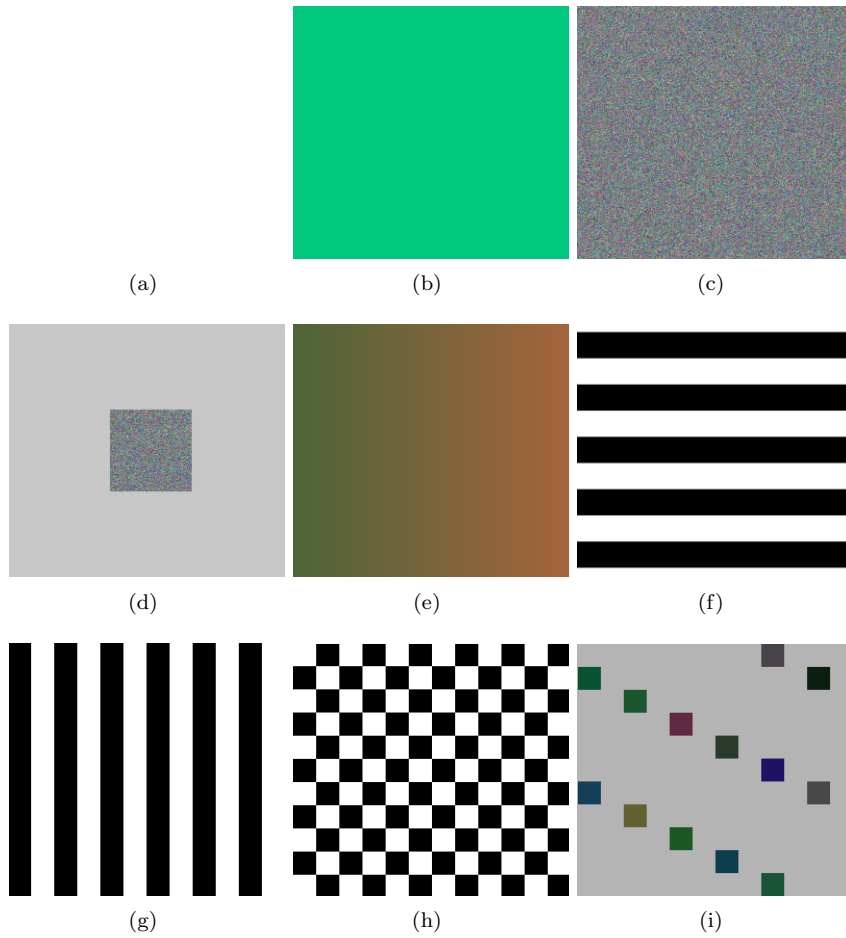


Figura 1: (a) Fondo plano idéntico entre canales (b) Fondo plano diferenciado entre canales (c) Ruido uniforme (d) Ruido uniforme sobre fondo plano (e) Gradiente sobre el canal azul (f) Líneas horizontales (g) Líneas verticales (h) Cuadrícula (i) Píxeles aleatorios aislados

A. Arquitectura del código

El diseño sigue un patrón de abstracción por capas, separando las responsabilidades en tres niveles. La Fig. 2 muestra el diagrama de clases del proyecto en formato UML.

A.1. Estructura general

Hay tres jerarquías de clases abstractas, cada una con funcionalidades diferentes:

- **ImageHandler**: Funciones de lectura y escritura de imágenes genéricas.
- **EncodedHandler**: Funciones de lectura y escritura de códigos genéricas.
- **Encoder**: Maneja la codificación y decodificación, usando las dos anteriores para el manejo de archivos.

Encoder no hereda de los handlers, sino que los compone como punteros, lo que permite intercambiarlos libremente. **PLREncoder** es la única implementación concreta del codificador.

A.2. Ventajas

- **Expansibilidad**: Para soportar PNG en lugar de BMP, basta con crear una clase `PNGHandler : public ImageHandler` sin tocar nada más. Lo mismo aplica para nuevos formatos de compresión, por ejemplo creando una clase crear una clase `LZ77 : public EncodedHandler`.
- **Bajo acoplamiento**: **PLREncoder** no conoce los detalles de lectura y escritura, solo trabaja con punteros a esas clases. Esto facilita el mantenimiento.
- **Separación de responsabilidades**: Cada clase tiene tareas definidas, **BMPHandler** maneja archivos *.bmp*, **PRLEncodedHandler** maneja el formato comprimido, y **PLREncoder** implementa el algoritmo.
- **Polimorfismo**: Al tipar los miembros `img` y `enc` como punteros a la clase base, se podría eventualmente cambiar la implementación en tiempo de ejecución sin modificar la clase.

A.3. Desventajas

- **Gestión manual de memoria**: **PLREncoder** aloca memoria en el constructor y la libera en el destructor. Esto es propenso a *memory leaks* si hay excepciones intermedias dentro de un proyecto mas grande.
- **Complejidad**: **EncodedData** contiene los metadatos de manera genérica, de manera tal de adaptarse a otras implementaciones de códigos. Esto presenta la desventaja de necesitar de un parseo mas complejo en distintas etapas del proceso. Además, esta estructura podría no representar bien todos los tipos de codificaciones posibles.

Referencias

- [James D. Murray, 2006] James D. Murray, W. V. R. (2006). Encyclopedia of graphics file formats: Run-length encoding. https://www.fileformat.info/mirror/egff/ch09_03.htm.
- [Liesch, 2003] Liesch, N. (2003). The bmp file format. http://www.ece.ualberta.ca/~elliott/ee552/studentAppNotes/2003_w/misc/bmp_file_format/bmp_file_format.htm.
- [Tamayo, 2005] Tamayo, G. R. (2005). Data compression guide: Run-length encoding. <https://sites.google.com/view/datacompressionguide/data-compression-preliminaries/run-length-encoding-rle?authuser=0>.